ITT 036 – Programming Assignment 1

Digital Signal Encoder and Decoder

Autumn 2026

Student:	Bhargav Koushal	**Language:**	Java
Visualization:	Java Swing and AWT		

Objective

Develop a modular Java application that generates digital bit streams, identifies the longest palindromic substring, implements line encoding and decoding schemes, and demonstrates analog-to-digital conversion using Pulse Code Modulation (PCM).

Implemented Features

- **Digital data generation:** Generates random binary sequences, sequences containing four or eight consecutive zeros, and accepts manually entered binary data.
- **Palindrome detection:** Finds the longest palindromic substring in the input data using the implemented palindrome detection algorithm.
- **Line coding:** Implements NRZ-L, NRZ-I, Manchester, Differential Manchester, Alternate Mark Inversion (AMI), B8ZS, and HDB3, with corresponding decoder classes.
- **PCM:** Generates sine or cosine samples or accepts custom samples, quantizes them into fixed-width binary codewords, combines the codewords into a bitstream, and reconstructs approximate sample values.
- **Visualization:** Uses Java Swing and AWT to display signal waveforms and compare original analog samples with reconstructed PCM samples.

Design and Assumptions

The program uses separate classes for data generation, palindrome detection, encoders, decoders, PCM processing, and visualization. Digital signals are represented as arrays of voltage-level samples. Decoders recover the bitstream by analysing the encoded signal samples rather than using the original input string.

PCM uses uniform quantization over a user-defined minimum and maximum range. Samples outside the specified range are clamped to the range. Each decoded sample is reconstructed using the midpoint of its quantization interval. Generated sine and cosine signals represent discrete samples of analog waveforms.

How to Run

1. Open the project in IntelliJ IDEA with a compatible JDK.
2. Ensure all Java source files are in the same package.
3. Run `Main.java` and select digital line coding or analog signal generation with PCM.
4. Run `AllDecodersTest.java` and `EdgeCaseTest.java` to repeat the round-trip tests.
5. Run `PCMTest.java` and `PCMPlotterTest.java` to verify PCM output and plots.

Testing and Verification

The initial round-trip test suite passed 49 out of 49 tests. The edge-case suite initially identified two B8ZS failures involving leading zero runs. After correcting the B8ZS decoder, the student reported that all 77 edge-case tests passed. The PCM visualization was also run successfully. These results apply to the tested inputs and do not establish correctness for every possible input.

References and AI Acknowledgment

- ITT 036, *Programming Assignment 1*, Autumn 2026, assignment handout.
- Oracle, *Java Platform Documentation*: `https://docs.oracle.com/en/java/`
- OpenAI ChatGPT was used for debugging suggestions and test development